Software Design Description (SDD)

VotaGT - Sistema de Gestión Electoral

Estándar de referencia: IEEE 1016-2009

Curso: Análisis de Sistemas II

Universidad: Universidad Mariano Gálvez de Guatemala

Versión: 1.0

Fecha: Octubre 2026

---

1. Propósito

El presente documento describe el diseño interno de los principales componentes de VotaGT.

Se documentan:

- Responsabilidades.

- Interfaces.

- Dependencias.

- Datos utilizados.

- Manejo de excepciones.

- Patrones de diseño aplicados.

VotaGT está implementado como un Monolito Modular utilizando Node.js y Express.

---

2. Organización general

La aplicación se encuentra organizada mediante las siguientes capas y componentes:

```
Routes

&#x20;  ↓

Middlewares

&#x20;  ↓

Controllers

&#x20;  ↓

Services

&#x20;  ↓

Repositories / Gateways / Strategies
```

```
&amp;#x20;  ↓

Persistencia
```

Los almacenes utilizados son:

- PostgreSQL.

- Redis.

- MongoDB.

- Neo4j.

---

3. Componente de Autenticación

3.1 Identificación

Nombre: Auth Module

Responsabilidad: Autenticación, autorización y obtención del perfil del usuario.

3.2 Archivos principales

```
src/routes/auth.routes.js

src/controllers/auth.controller.js

src/services/auth.service.js

src/repositories/auth.repository.js

src/middlewares/auth.middleware.js

src/middlewares/permission.middleware.js
```

3.3 Interfaces

POST /api/v1/auth/login

Permite autenticar un usuario.

Entrada:

- identificador.

- password.

Salida:

- Token JWT.

- Información básica del usuario.

GET /api/v1/auth/perfil

Obtiene la información del usuario autenticado.

Requiere:

`Authorization: Bearer <token>`

3.4 Dependencias

El componente depende de:

- PostgreSQL.

- bcrypt.

- jsonwebtoken.

- EventBus.

- Neo4j Authorization Graph.

3.5 Comportamiento

El servicio:

1. Valida los datos recibidos.

2. Obtiene el usuario desde PostgreSQL.

3. Comprueba el estado del usuario.

4. Compara la contraseña mediante bcrypt.

5. Obtiene roles y permisos.

6. Genera un JWT.

7. Sincroniza la autorización con Neo4j.

8. Publica el evento `LOGIN\_EXITOSO`.

3.6 Excepciones

Puede generar:

- HTTP 400 por datos incompletos.

- HTTP 401 por credenciales inválidas.

- HTTP 403 cuando la cuenta está bloqueada.

- HTTP 500 por errores internos no controlados.

Las credenciales inválidas utilizan un mensaje genérico para evitar revelar información sensible.

---

4. Componente de Votaciones

4.1 Identificación

Nombre: Votacion Module

Responsabilidad: Gestión de procesos electorales.

4.2 Archivos

```
src/routes/votacion.routes.js

src/controllers/votacion.controller.js

src/services/votacion.service.js

src/repositories/votacion.repository.js
```

4.3 Interfaces

GET /api/v1/votaciones

Lista las votaciones registradas.

POST /api/v1/votaciones

Crea una nueva votación.

GET /api/v1/votaciones/{votacionId}

Obtiene una votación específica.

4.4 Dependencias

- PostgreSQL.
- Middleware JWT.
- Middleware de permisos.
- EventBus.

4.5 Reglas

La fecha de finalización deberá ser posterior a la fecha de inicio.

El nombre de la votación es obligatorio.

4.6 Eventos

Después de una creación exitosa se publica:

`VOTACION\_CREADA`

El Audit Observer recibe este evento y lo almacena en MongoDB.

4.7 Excepciones

- HTTP 400: datos inválidos.
- HTTP 404: votación inexistente.
- HTTP 401: usuario no autenticado.
- HTTP 403: permiso insuficiente.

---

5. Componente de Candidatos

5.1 Identificación

Nombre: Candidato Module

Responsabilidad: Registrar y consultar candidatos de una votación.

5.2 Archivos

```
src/routes/candidato.routes.js

src/controllers/candidato.controller.js

src/services/candidato.service.js

src/repositories/candidato.repository.js
```

5.3 Interfaces

GET /api/v1/votaciones/{votacionId}/candidatos

Lista los candidatos de una votación.

POST /api/v1/votaciones/{votacionId}/candidatos

Registra un candidato.

5.4 Atributos principales

- nombre.
- partido.
- cargo.
- numeroLista.
- votacionId.

5.5 Reglas

- Nombre obligatorio.
- Cargo obligatorio.
- Número de lista entero positivo.
- Número de lista único dentro de la votación.

5.6 Dependencias

- PostgreSQL.
- Candidato Repository.
- EventBus.
- Middleware de autorización.

5.7 Eventos

`CANDIDATO\_CREADO`

5.8 Excepciones

- HTTP 400: datos inválidos.

- HTTP 404: votación inexistente.

- HTTP 409: número de lista duplicado.

---

6. Componente de Mesas

6.1 Identificación

Nombre: Mesa Module

Responsabilidad: Administración de mesas electorales.

6.2 Archivos

```
src/routes/mesa.routes.js

src/controllers/mesa.controller.js

src/services/mesa.service.js

src/repositories/mesa.repository.js
```

6.3 Interfaces

GET /api/v1/votaciones/{votacionId}/mesas

Lista las mesas de una votación.

POST /api/v1/votaciones/{votacionId}/mesas

Registra una nueva mesa.

6.4 Atributos

- codigo.

- nombre.

- ubicacion.

- departamento.

- municipio.

- votacionId.

6.5 Reglas

- Código obligatorio.

- Nombre obligatorio.

- El código no deberá repetirse dentro de una misma votación.

6.6 Dependencias

- PostgreSQL.

- Mesa Repository.

- EventBus.

- Middleware de permisos.

6.7 Eventos

`MESA\_CREADA`

6.8 Excepciones

- HTTP 400: datos inválidos.

- HTTP 404: votación inexistente.

- HTTP 409: código duplicado.

---

7. Componente de Resultados

7.1 Identificación

Nombre: Resultado Module

Responsabilidad: Registrar, consultar y exportar resultados electorales.

7.2 Archivos

```
src/routes/resultado.routes.js

src/controllers/resultado.controller.js

src/services/resultado.service.js

src/repositories/resultado.repository.js
```

7.3 Interfaces

POST /api/v1/votaciones/{votacionId}/resultados

Registra votos para un candidato en una mesa.

GET /api/v1/votaciones/{votacionId}/resultados

Obtiene los resultados consolidados.

GET /api/v1/votaciones/{votacionId}/resultados/exportar

Exporta resultados.

Parámetro:

`formato=json`

o:

`formato=csv`

7.4 Atributos principales

- votacionId.

- mesaId.

- candidatoId.

- votos.

- registradoPor.

7.5 Reglas

Los votos deberán ser enteros mayores o iguales a cero.

La mesa deberá pertenecer a la votación.

El candidato deberá pertenecer a la votación.

No podrá existir más de un resultado para la misma combinación:

```
mesa + candidato
```

7.6 Dependencias

- Resultado Repository.

- Cache Gateway.

- EventBus.

- Strategy de exportación.

- PostgreSQL.

- Redis.

7.7 Eventos

Después del registro se publica:

`RESULTADO\_REGISTRADO`

7.8 Excepciones

- HTTP 400: votos o identificadores inválidos.

- HTTP 404: votación, mesa o candidato inexistente.

- HTTP 409: resultado duplicado.

---

8. Componente Repository

8.1 Responsabilidad

Repository Pattern abstrae el acceso a PostgreSQL.

Los servicios no ejecutan directamente consultas SQL.

8.2 Implementaciones

```
auth.repository.js

votacion.repository.js

candidato.repository.js

mesa.repository.js

resultado.repository.js
```

8.3 Interfaz conceptual

Los repositorios exponen operaciones como:

```
crear

consultar

listar

buscar

verificar existencia
```

8.4 Beneficio

La lógica de negocio permanece desacoplada del mecanismo de persistencia.

---

9. Cache Gateway

9.1 Identificación

Archivo:

```
src/gateways/cache.gateway.js
```

9.2 Responsabilidad

Encapsular el acceso a Redis.

9.3 Operaciones

```
obtener(clave)

guardar(clave, datos, ttl)

eliminar(clave)

existe(clave)
```

```
obtenerEstadoCircuito()
```

9.4 Dependencias

- Redis Client.

- Circuit Breaker.

9.5 Manejo de fallos

Cuando Redis no está disponible, el Gateway devuelve un fallback controlado.

Para lectura:

```
null
```

Esto permite que ResultadoService consulte PostgreSQL.

---

10. Circuit Breaker

10.1 Identificación

Archivo:

```
src/patterns/circuitBreaker.js
```

10.2 Responsabilidad

Evitar intentos continuos hacia Redis cuando el servicio se encuentra fuera de línea.

10.3 Atributos

```
nombre

limiteFallos

tiempoReintento

fallos

estado

proximoIntento
```

10.4 Estados

```
CLOSED

OPEN
```

```
HALF\_OPEN
```

10.5 Operación principal

```
ejecutar(operacion, fallback)
```

10.6 Comportamiento

Cuando el número de errores alcanza el límite configurado:

```
CLOSED -> OPEN
```

Después del periodo de recuperación:

```
OPEN -> HALF\_OPEN
```

Si el nuevo intento funciona:

```
HALF\_OPEN -> CLOSED
```

10.7 Excepciones

Cuando existe un fallback, el error del servicio auxiliar se controla sin propagar una falla hacia la operación principal.

---

11. Componente de Auditoría

11.1 Elementos

```
src/events/eventBus.js

src/events/audit.observer.js

src/audit/audit.service.js
```

11.2 Responsabilidad

Registrar eventos relevantes sin acoplar los servicios directamente con MongoDB.

11.3 Flujo

```
Service

&#x20;  ↓
```

```
eventBus.emit("AUDITORIA")

&#x20;  ↓

Audit Observer

&#x20;  ↓

Audit Service

&#x20;  ↓

MongoDB
```

11.4 Interfaz de evento

Un evento puede contener:

```
tipo

usuarioId

username

recurso

recursoId

detalle

ip
```

11.5 Manejo de errores

Un error de auditoría no deberá impedir que la operación principal del sistema se complete.

El Audit Observer captura los errores y los registra en consola.

---

12. Componente Strategy de Resultados

12.1 Archivo

```
src/strategies/resultados.strategy.js
```

12.2 Responsabilidad

Permitir diferentes algoritmos de generación de reportes sin modificar el servicio principal.

12.3 Estrategias

```
JsonResultadosStrategy

CsvResultadosStrategy
```

12.4 Operación común

```
generar(datos)
```

Cada estrategia devuelve:

```
contenido

contentType

extension
```

12.5 Selección

La función:

```
obtenerEstrategia(formato)
```

selecciona la implementación correspondiente.

12.6 Excepciones

Un formato diferente de:

```
json

csv
```

produce:

```
HTTP 400

invalid-report-format
```

---

13. Componente de autorización Neo4j

13.1 Archivo

```
src/graph/authorization.graph.js
```

13.2 Responsabilidad

Sincronizar una representación gráfica de las relaciones entre usuarios, roles y permisos.

13.3 Modelo

```
Usuario

&#x20;  |

TIENE\_ROL

&#x20;  |

&#x20;  v

Rol

&#x20;  |

OTORGA

&#x20;  |

&#x20;  v

Permiso
```

13.4 Dependencia

Neo4j Driver.

13.5 Uso

La sincronización se realiza durante el proceso de autenticación después de obtener roles y permisos.

---

14. Middleware de autenticación

14.1 Archivo

```
src/middlewares/auth.middleware.js
```

14.2 Responsabilidad

Validar el token JWT enviado mediante:

```
Authorization: Bearer <token>
```

14.3 Excepciones

Sin token válido:

```
HTTP 401
```

---

15. Middleware de permisos

15.1 Archivo

```
src/middlewares/permission.middleware.js
```

15.2 Responsabilidad

Validar que un usuario autenticado posea el permiso necesario antes de ejecutar una operación.

Ejemplo:

```
REPORTES\_GENERAR
```

15.3 Excepciones

Usuario no autenticado:

```
HTTP 401
```

Usuario autenticado sin permiso:

```
HTTP 403
```

---

16. Error Handler

16.1 Archivo

```
src/middlewares/errorHandler.js
```

16.2 Responsabilidad

Centralizar el tratamiento de excepciones de la API.

16.3 Formato

Los errores son devueltos mediante:

```
application/problem+json
```

con campos:

```
type

title

status

detail

instance
```

16.4 Seguridad

Los errores HTTP 500 no deberán revelar información interna sensible al cliente.

---

17. Dependencias entre componentes

El flujo principal puede representarse así:

```
Route

&#x20; ↓

Middleware

&#x20; ↓

Controller

&#x20; ↓

Service

&#x20; ├──────────> Repository ─────> PostgreSQL

&#x20; │

&#x20; ├──────────> Cache Gateway ──> Circuit Breaker ──> Redis

&#x20; │

&#x20; ├──────────> EventBus ────────> Observer ──────────> MongoDB
```

```
&amp;#x20; │
&amp;#x20; ├──────────> Strategy
&amp;#x20; │
&amp;#x20; └──────────> Authorization Graph ──────────────────> Neo4j
```

---

18. Patrones aplicados

Patrón	Componente	Propósito
Repository	Repositories	Abstraer PostgreSQL
Gateway	Cache Gateway	Abstraer Redis
Observer	EventBus / Audit Observer	Desacoplar auditoría
Circuit Breaker	Redis	Tolerancia a fallos
Strategy	Exportación	Seleccionar JSON o CSV

---

19. Decisiones de diseño

El diseño busca mantener una alta cohesión dentro de cada módulo y reducir el acoplamiento entre las capas.

Los controladores no contienen lógica de persistencia.

Los servicios no ejecutan SQL directamente.

El acceso a Redis se encuentra encapsulado.

La auditoría se encuentra desacoplada mediante eventos.

La generación de reportes utiliza estrategias intercambiables.

Las excepciones HTTP son procesadas mediante un middleware centralizado.

---

20. Conclusión

El diseño de VotaGT divide las responsabilidades mediante componentes claramente definidos.

La combinación de Services, Repositories, Gateways, Observer, Strategy y Circuit Breaker permite mantener una estructura modular, comprobable y preparada para evolución futura.

El diseño documentado corresponde a los componentes implementados actualmente en el backend del sistema.